

adoctor-check 异常检测模块

adoctor-check 异常检测模块

1 介绍

adoctor-check 异常检测模块主要是通过对收集到的集群内机器节点上各类运行数据进行计算与分析，来确认数据项是否存在异常的工具。

2 架构

异常检测工具依托于 aops 框架运行，主要分为 scheduler 和 executor 两个模块。

图片说明：异常检测模块架构

3 软件下载

- Repo 源挂载正式发布地址：
- 源码获取地址：
- RPM 包版本获取地址：

4 运行环境

- 硬件配置
- 配置项：CPU；推荐规格：8 核
- 配置项：内存；推荐规格：32G，最小 4G
- 配置项：网络带宽；推荐规格：300M
- 配置项：I/O；推荐规格：375MB/sec
- 软件配置
- 软件名：Python；版本和规格：版本 3.8 及以上

- 软件名：mysql；版本和规格：8.0.26
- 软件名：kafka；版本和规格：2.6.0
- 软件名：zookeeper；版本和规格：3.6.2
- 软件名：elasticsearch；版本和规格：7.14.0
- 软件名：prometheus；版本和规格：2.20.0
- 软件名：node_exporter；版本和规格：1.0.1
- 软件名：aops-database；版本和规格：1.0.1
- 软件名：aops-manager；版本和规格：1.0.1
- 软件名：fluentd；版本和规格：1.13.3
- 软件名：fluentd-plugin-elasticsearch；版本和规格：5.0.5

5 安装工具

5.1 Aops 框架安装

异常检测模块运行依赖于 aops 框架，框架的安装步骤参见服务部署手册

5.2 依赖组件安装

异常检测模块依赖消息中间件与数据采集模块，需要保证环境中部署组件 zookeeper, kafka, prometheus, node_exporter, elasticsearch, fluentd。其中 elasticsearch 在 aops 框架安装中会进行部署，需要注意修改相关配置，适配多主机的 fluentd 采集数据推送。

5.2.1 使用 Aops 部署服务安装

adoctor-check-scheduler, adoctor-check-executor 可以与本步骤合并操作。

5.2.1.1 编辑任务列表

修改部署任务列表，打开 zookeeper, kafka, prometheus, node_exporter, fluentd 组件步骤的开关：

step_list:

```
zookeeper:
  enable: true
  continue: false
kafka:
  enable: true
  continue: false
prometheus:
  enable: true
  continue: false
node_exporter:
  enable: true
  continue: false
fluentd:
  enable: true
  continue: false
...
```

5.2.1.2 编辑主机清单

具体步骤参见自动化部署服务手册章节 2.2.2 章节各组件主机配置

5.2.1.3 编辑变量列表

具体步骤参见自动化部署服务手册章节 2.2.2 章节各组件变量配置

5.2.1.4 执行部署任务

1. 前端执行：

参见服务部署手册 1.8.2 章节

2. 命令行执行：

- 登录待安装的服务器后台
- 执行命令：

```
aops task --action execute --task_list a --access_token token
```

5.2.2 手动安装

以上依赖组件可以配置 openeuler 的 repo 源安装，具体的参数配置与集群搭建可参见各软件的官网文档。

5.3 adoctor-check 安装

5.3.1 手动安装

- 通过 dnf 挂载 repo 源实现

先使用 dnf 挂载 adoctor-check-scheduler 和 adoctor-check-executor 软件在所在 repo 源（具体方法可参考应用开发指南），然后执行如下指令下载以及安装 pkgship 及其依赖。

```
dnf install adoctor-check-scheduler
dnf install adoctor-check-executor
```

- 通过安装 rpm 包实现。先下载 adoctor-check-scheduler, adoctor-check-executor, aops-utils，然后执行如下命令进行安装（其中 x.x-x 表示版本号，请用实际情况替代）

```
rpm -ivh adoctor-check-scheduler-x.x-x.oe1.noarch.rpm
rpm -ivh adoctor-check-executor-x.x-x.oe1.noarch.rpm
```

5.3.2 使用 Aops 部署服务安装

可以与 5.2.1 章节依赖组件合并任务操作。

5.3.2.1 编辑任务列表

修改部署任务列表，打开 adoctor-check-scheduler, adoctor-check-executor 步骤开关：

```
---
step_list:
...
adoctor_check_executor:
  enable: true
  continue: false
adoctor_check_scheduler:
  enable: true
  continue: false
...
```

5.3.2.2 编辑主机清单

具体步骤参见自动化部署服务手册章节 2.2.2.8 章节 check 模块主机配置

5.3.2.3 编辑变量列表

具体步骤参见自动化部署服务手册章节 2.2.2.8 章节 check 模块变量配置

5.3.2.4 执行部署任务

参见 章节 5.2.1.4 执行部署任务

6 异常检测规则定义

异常检测规则根据系统可采集的数据进行定义。

6.1 异常检测规则文件格式

异常检测规则需要以 json 格式导入，不同用户可以导入不同的检测规则。检测规则列表由检测项组成。检测项的实例如下所示：

```
"check_items": [  
  {  
    "check_item": "check_item2",  
    "data_list": [  
      {  
        "name": "data1",  
        "type": "kpi",  
        "label": {  
          "cpu": "1",  
          "mode": "irq"  
        }  
      }  
    ],  
    "condition": "$0>1",  
    "plugin": "",  
    "description": "data 1"  
  },  
  {  
    "check_item": "check_item2",  
    ...  
  }  
]
```

异常检测规则主要包含如下几个字段：

- 字段名称: check_item; 含义: 异常检测规则名称; 类型: string; 备注: 一个用户的异常检测规则名称唯一
- 字段名称: data_list; 含义: 本项异常检测所需要的数据列表; 类型: list
- 字段名称: condition; 含义: 检测规则描述; 类型: string; 备注: 字符串中描述异常的情况。描述 data_list 中的数据在满足什么条件的情况下视为某种异常
- 字段名称: plugin; 含义: 异常检测规则插件名称; 类型: string; 备注: 如果选择使用自定义的异常检测规则插件, 此处填写插件名称。填写空字符串, 则默认会使用内置的表达式插件解析检测规则。
- 字段名称: description; 含义: 异常检测项描述; 类型: string

6.2 异常检测数据项说明

每一个异常检测项都需要描述其计算所需要的数据, 在 data_list 中列出。data_list 的主要字段说明

- 字段名称: name; 含义: 数据项名称; 类型: string; 备注: 在一个 data_list 中数据项名称是唯一的
- 字段名称: type; 含义: 数据类型; 类型: string; 备注: 可选 kpi (指标类数据) 或 log (日志类数据)
- 字段名称: label; 含义: 数据标签; 类型: dict; 备注: 字典中记录数据项的键值对, 需要完整描述一项数据的所有 label。具体取值 kpi 类数据可以参考 node_exporter 的 metrics 列表, log 类数据可以不带 label 字段
- kpi 类数据

kpi 类数据当前主要由 node_exporter 采集, 存储在 prometheus 中。不同的 node_exporter 版本可以采集的数据指标会有一些差异, 需以实际部署的 node_exporter 版本支持的数据项为准。

– 数据类型

主要包括以下类目:

- 数据项: node_cpu*; 数据类型: 系统 cpu 相关指标
- 数据项: node_disk*; 数据类型: 磁盘 IO 指标
- 数据项: node_filesystem*; 数据类型: 文件系统相关指标

- 数据项：node_memory*、node_zoneinfo*；数据类型：系统内存使用相关指标
- 数据项：node_netstat*、node_network*、node_sockstat*；数据类型：网络相关指标
- 数据项：node_load*；数据类型：系统负载相关指标
- 数据项：node_systemd*；数据类型：service 相关指标
- 数据项：node_time*；数据类型：时间相关指标
- 数据项：process*；数据类型：prometheus 自身进程相关指标
- 数据项：go_*；数据类型：go 环境相关指标

– 数据标签

kpi 数据通常会带有能够表征自身特点的标签，在 label 中以字典的形式描述。一组标签可以唯一地标识一条指标数据。标签的维度可以是该监控数据的状态、特征、环境定义等。例如：

```
node_cpu_seconds_total{cpu="0",mode="idle"}
node_cpu_seconds_total{cpu="0",mode="iowait"}
node_cpu_seconds_total{cpu="0",mode="irq"}
node_cpu_seconds_total{cpu="0",mode="nice"}
node_cpu_seconds_total{cpu="0",mode="softirq"}
node_cpu_seconds_total{cpu="0",mode="steal"}
node_cpu_seconds_total{cpu="0",mode="system"}
node_cpu_seconds_total{cpu="0",mode="user"}
...
node_cpu_seconds_total{cpu="95",mode="softirq"}
node_cpu_seconds_total{cpu="95",mode="steal"}
node_cpu_seconds_total{cpu="95",mode="system"}
node_cpu_seconds_total{cpu="95",mode="user"}
```

上面的一组数据中，每一项数据有的 cpu 和 mode 标签。cpu 标签表述所属 cpu 核，共 96 个，mode 表示 cpu 的空闲状态(idle)，用户空间进程使用状态(user)，系统空间进程使用状态(system)等 8 种状态。

node_exporter 支持的数据项的详细说明参见 node_exporter 官网文档说明

- log 类数据

日志类数据主要是由 fluentd 采集，存储在 elasticsearch 中。目前支持采集的数据包括系统的 history 日志和 demesg 日志。

6.3 异常检测规则条件说明

6.3.1 异常检测表达式

异常检测过程的规则中默认使用表达式来创建关于采集数据的表示。常规的表达式如下：

expression 是包含数据变量的表达式。数据变量来自于 data_list 中的描述，并且按照在 data_list 中的数据列表顺序以 \$i 宏来表述。

举例，若 data_list 为：

```
"data_list":
[
  {
    "name": "node_cpu_frequency_min_hertz",
    "type": "kpi",

  },
  {
    "name": "node_cpu_guest_seconds_total",
    "type": "kpi",
    "label": {
      "cpu": "1",

    }
  },
  {
    "name": "node_cpu_seconds_total",
    "type": "kpi"
  }
]
```

则 condition 应类似如下描述：

$\$0 + \$1 * \$2 > 100$

其计算时的含义为：

$\text{node_cpu_frequency_min_hertz} + \text{node_cpu_guest_seconds_total} * \text{node_cpu_seconds_total} > 100$

6.3.2 运算符

当前支持以下运算符

- 优先级：1；运算符：（）；定义：圆括号；使用形式：(expression)、function(parameter)
- 优先级：2；运算符：-；定义：负号运算符；使用形式：-expression
- 运算符：+；定义：正号运算符；使用形式：+expression
- 优先级：+；运算符：!；定义：逻辑非运算符；使用形式：!expression
- 运算符：~；定义：按位取反运算符；使用形式：~expression
- 优先级：3；运算符：/；定义：除；使用形式：expression1 / expression2
- 运算符：*；定义：乘；使用形式：expression1 * expression2
- 运算符：%；定义：余数(取模)；使用形式：expression1 % expression2
- 优先级：4；运算符：+；定义：加；使用形式：expression1 + expression2
- 运算符：-；定义：减；使用形式：expression1 - expression2
- 优先级：5；运算符：>；定义：右移；使用形式：expression1 >> expression2
- 优先级：6；运算符：>；定义：大于；使用形式：expression1 > expression2
- 运算符：>=；定义：大于等于；使用形式：expression1 >= expression2
- 运算符：.function()

针对\$0 数据进行 function 运算后与 constant 作比较

function()

- 函数参数

函数中支持时间与个数作为过滤器。

- 函数调用：max(30s)；含义：30 秒内数据的最大值
- 函数调用：max(#5)；含义：最近 5 个数据的最大值

时间偏移量支持单位符号 s(秒)，m(分)，h(时)

- 函数列表

- 函数名: count(sec|#num, pattern, operator); 描述: 计算指定一段数据中符合条件的数据个数; 参数: sec|#num: 时间或个数的偏移量 \: 判断的对比项 \: 判断的关系; 数据类型: float, int, str; 备注: count(10m, 1, ">"): 十分钟内大于 1 的数据的个数
count(#15, "error", "=="): 最近 15 个数据中等于 error 的个数
- 函数名: max(sec|#num); 描述: 计算指定一段数据的最大值; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int; 备注: max(#5): 最近 5 个数的最大值 max(5s): 最近 5 秒内数据的最大值
- 函数名: min(sec|#num); 描述: 计算指定一段数据的最小值; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int; 备注: min(#5): 最近 5 个数的最小值 min(5s): 最近 5 秒内数据的最小值
- 函数名: sum(sec|#num); 描述: 计算指定一段数据的总和; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int; 备注: sum(#5): 最近 5 个数的总和 sum(5s): 最近 5 秒内数据的总和
- 函数名: avg(sec|#num); 描述: 计算指定一段数据的平均值; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int; 备注: avg(#5): 最近 5 个数的平均值 avg(5s): 最近 5 秒内数据的平均值
- 函数名: keyword(pattern); 描述: 判断当前数据中是否包含指定关键字字符串; 参数: pattern: 关键字符; 数据类型: str; 备注: keyword("error"): 数据中包含 error 关键字返回 true, 否则返回 false
- 函数名: diff(sec|#num); 描述: 当前值与之前的值是否相同; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int, str; 备注: true: 两个值相同 false: 两个值不同
- 函数名: abschange(sec|#num); 描述: 当前值与之前的值差值的绝对值; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int; 备注: pre:1 cur:5 return: 4 pre:3 cur:1 return: 2 pre:0 cur:-2.5 return 2.5
- 函数名: change(sec|#num); 描述: 当前值与之前的值差值; 参数: sec|#num: 时间或个数的偏移量; 数据类型: float, int; 备注: pre:1 cur:5 return: 4 pre:3 cur:1 return: -2 pre:0 cur:-2.5 return -2.5

6.4 自定义异常检测插件

- 自定义异常检测插件接口

自定义异常检测插件需要继承
adoctor_check_executor.check_rule_plugins.check_rule_plugin 中的 CheckRulePlugin 类。

- 接口: judge_condition; 功能: 判断是否有异常; 输入值: index: 在时序数据段中的坐标 data_vector: 时序数据段 main_data_name: 主数据; 返回值: true: 存在异常 false: 无异常
- 功能: 设置插件管理器; 输入值: plugin_manager: 插件管理器

- 配置插件文件

在/etc/aops/check_rule_plugin.yml 中添加自己的自定义异常检测规则插件：

```
```yaml
```

```

```

```
plugin_list:
```

- plugin\_name: expression\_rule\_plugin  
file\_name: expression\_rule  
module\_name: ExpressionCheckRule
- plugin\_name: my\_plugin #插件名  
file\_name: my\_plugin #插件文件  
module\_name: Myplugin #插件类名

1. 将 my\_plugin.py 以及需要引用的文件移动到 adoctor\_check\_executor 的安装目录下：adoctor\_check\_executor/check\_rule\_plugins
2. 重启 adoctor-check-executor，将会加载自定义插件

## 7 配置参数

### 7.1 配置 adoctor-check-scheduler

adoctor-check-scheduler 默认的配置文件存放在/etc/aops/check\_scheduler.ini 中，请根据实际情况修改：

```
vim /etc/aops/check_scheduler.ini
```

```
; kafka producer 相关参数
```

```
[producer]
```

```
; kafka 服务器列表，为 host:port 组成的列表，端口默认是 9092，IP 默认取 localhost
```

```
kafka_server_list = 192.168.1.1:9092
```

```
; kafka API 的版本，默认取 0.11.5
```

```
api_version = 0.11.5
```

```
; kafka 的 leader 在确认请求完成前收到的 ack 数量。默认为 1。
```

```
; 0: producer 不会等待服务器的 ack。消息会立即添加到套接字缓冲区，并置为已发送。不保证服务器已经收到消息，每个消息返回的偏移量始终为-1。
```

```
; 1: leader 只将记录写入本地。broker 不会等待所有的 follower 返回响应。
```

```
; all: 等待所有的同步副本都写入。
acks = 1
;取值大于 0 时, 当客户端发送失败时会重新发送
retries = 3
; 重试错误时后退的毫秒数
retry_backoff_ms = 100

; kafka consumer 相关参数
[consumer]
; kafka 服务器列表, 为 host:port 组成的列表, 端口默认是 9092, IP 默认取 localhost
kafka_server_list = 192.168.1.1:9092
; 如果为 True, consumer 的 offset 会在后台定期 commit
enable_auto_commit = False
; 针对 OffsetOutOfRange 错误的 offset 重置策略。
; earliest: 移动到最早的可用的消息
; latest: 移动到最新的消息
auto_offset_reset = earliest
; consumer 进行 poll 操作时, 如果缓冲区没有数据, 在轮询中等待的时间。取 0 则立即返回缓冲区中当前的可用记录。否则返回空
timeout_ms = 5
; consumer 进行 poll 操作时, 单次调用返回的最大记录数
max_records = 3

; check scheduler 的相关配置参数
[check_scheduler]
; check scheduler 监听的地址
ip = 127.0.0.1
; check scheduler 监听的端口
port = 11112
; 异常检测过程中出现无数据或内部错误的异常情况时, 触发任务重试的最大次数
max_retry_num = 3
; 重试一次后仍然失败, 等待下一次重试的冷却时间
cool_down_time = 120
; 超过最大重试次数的任务, 即死亡任务, 任务 ID 在缓存中记录个数。超过之后会清理缓存
max_dead_retry_task = 10000
; 清理死亡任务时的清除比例。默认配置下, 即超过 10000 时, 默认删掉 50%的缓存
dead_retry_task_discount = 0.5
; 从启动 check scheduler 开始后向任务 (逆时间轴) 的检测时间步长
backward_task_step = 60
; 触发后向任务的时间间隔
```

```

backward_task_interval = 30
; 触发前向任务（顺时间轴）的时间间隔
forward_task_interval = 30
; 触发前向任务的最大时间间隔
forward_max_task_step = 86400

; uwsgi 的相关配置参数
[uwsgi]
; uwsgi 的启动脚本
wsgi-file=manage.py
; check uwsgi 服务的日志
daemonize=/var/log/aops/uwsgi/check_scheduler.log
; HTTP 连接超时时间
http-timeout=600
; 服务器响应时间
harakiri=600

```

注意，check\_scheduler 服务监听的 IP 和 Port 修改后，/etc/aops/system.ini 中 check\_scheduler 的 IP 与 Port 需要同步修改。

## 7.2 配置 adocor-check-executor

### 7.2.1 基本参数配置

adocor-check-executor 默认的配置文件存放在/etc/aops/check\_executor.ini 中，请根据实际情况修改：

```
vim /etc/aops/check_executor.ini
```

```

; check scheduler 的相关配置参数
[consumer]
; kafka 服务器列表，为 host:port 组成的列表，端口默认是 9092，IP 默认取 localhost
kafka_server_list=192.168.1.1:9092
; 如果为 True, consumer 的 offset 会在后台定期 commit
enable_auto_commit=False
; 针对 OffsetOutOfRange 错误的 offset 重置策略。
; earliest: 移动到最早的可用的消息
; latest: 移动到最新的消息
auto_offset_reset=earliest
; consumer 进行 poll 操作时，如果缓冲区没有数据，在轮询中等待的时间。取 0 则立即返回缓冲区中当前的可用记录。否则返回空
timeout_ms=5

```

```
; consumer 进行 poll 操作时，单次调用返回的最大记录数
max_records=3

; kafka producer 相关参数
[producer]
; kafka 服务器列表，为 host:port 组成的列表，端口默认是 9092，IP 默认取 localhost
kafka_server_list=192.168.1.1:9092
; kafka API 的版本，默认取 0.11.5
api_version=0.11.5
; kafka 的 leader 在确认请求完成前收到的 ack 数量。默认为 1。
; 0: producer 不会等待服务器的 ack。消息会立即添加到套接字缓冲区，并置为已发送。不保证服务器已经收到消息，每个消息返回的偏移量始终为-1。
; 1: leader 只将记录写入本地。broker 不会等待所有的 follower 返回响应。
; all: 等待所有的同步副本都写入。
acks=1
; 取值大于 0 时，当客户端发送失败时会重新发送
retries=3
; 重试错误时后退的毫秒数
retry_backoff_ms=100

; check executor 相关配置参数
[executor]
; 插件加载的相对路径，从 adocor_check_executor 的 Python 包引用相对路径
plugin_path = adocor_check_executor.check_rule_plugins
; 执行具体异常检测的 consumer 数量
do_check_consumer_num = 2
; 采集数据的采样周期，需要与 prometheus 的 scrape 周期配置一致，否则检测结果将不准确
sample_period = 15
```

## 7.2.2 自定义异常检测规则插件配置

/etc/aops/check\_rule\_plugin.yml 中配置了异常检测的规则插件，默认配置内置的 expression\_rule\_plugin，可以根据章节 6.4 中添加自定义插件的配置。

# 8 异常检测规则管理

## 8.1 导入异常检测规则

### 8.1.1 前端操作

1. 准备异常检测规则 json 文件。

2. 打开规则管理页面，新建异常检测规则。

图片说明：导入异常检测规则

### 8.1.2 命令行操作

1. 登录到部署 adocor-check-scheduler 的服务器后端。
2. 准备异常检测规则 json 文件，拷贝至服务器环境上。
3. 则输入命令：

```
adocor checkrule [--action] add [--conf] [check.json] [--access_token] [token]
```

参数说明：

- action: 取 add 表示添加操作
  - conf: 待导入的规则文件
  - access\_token: 访问令牌
4. 举例，导入 check\_rule.json 文件，则输入命令：

```
adocor checkrule --action add --conf check_rule.json --access_token "1111"
```

## 8.2 删除异常检测规则

### 8.2.1 前端操作

1. 打开规则管理页面
2. 在规则列表中删除规则

图片说明：删除异常检测规则

### 8.2.2 命令行操作

1. 登录到部署 adocor-check-scheduler 的服务器后端。
2. 输入命令：

```
adocor checkrule [--action] delete [--check_items] [items] [--access_token] [token]
```

参数说明：

- action: 取 delete 表示删除操作
- check\_items: 表示删除指定的检测项，以逗号分隔，为空时不删除任何检测项
- access\_token: 访问令牌

3. 举例，删除检测项 `cpu_usage_overflow`，输入命令：

```
adoctor checkrule --action delete --check_items cpu_usage_overflow --
access_token "1111"
```

## 8.3 获取异常检测规则

### 8.3.1 前端操作

1. 打开异常检测规则列表，查看异常检测规则。

图片说明：获取异常检测规则

### 8.3.2 命令行操作

1. 登录到部署 `adoctor-check-scheduler` 的服务器后端。
2. 输入命令：

```
adoctor checkrule [--action] get [--check_items] [items] [--export] [path] [--sort]
[check_item] [--direction] [{asc, desc}] [--page] [page] [--per_page] [per_page] [--
access_token] [token]
```

参数说明：

- `action`: 取 `get` 表示获取操作
  - `check_items`: 表示获取指定的检测项，以逗号分隔，为空时表示获取全部检测项
  - `sort`: 对异常检测结果进行排序，目前可选 `start` 或 `end` 或 `check_item`，为空表示不排序
  - `direction`: 异常结果排序可选升序(`asc`)，降序(`desc`)，默认:`asc`
  - `page`: 当前的页码
  - `per_page`: 每页的数量，最大为 50
  - `access_token`: 访问令牌
3. 举例，导出检测项 `cpu_usage_overflow`，仅获取第 1 页，每页显示 30 个，按 `check_item` 升序排序，保存到 `/tmp` 目录：

```
adoctor checkrule --action get --check_items cpu_usage_overflow --export /tmp --sort
check_item --direction asc --page 1 --per_page 30 --access_token "1111"
```



## 9 异常检测功能使用

### 9.1 异常检测执行

在 adocor-check-scheduler 与 adocor-check-executor 服务启动之后，即会从启动时间开始进行实时异常检测，以及历史时间的检测，历史时间的检测会按照配置的时间步长，逐步回溯。

### 9.2 获取异常检测结果

#### 9.2.1 前端操作

1. 查看异常检测结果列表。

图片说明：获取异常检测结果

2. 查看异常检测结果统计。

图片说明：异常检测结果统计

#### 9.2.2 命令行操作

1. 登录到部署 adocor-check-scheduler 的服务器后端。
2. 输入命令：

```
adocor check [--check_items] [items] [--host_list] [list] [--start] [time1] [--end] [time2] [--sort] [{check_item, start, end}] [--direction] [{asc, desc}] [--page] [page] [--per_page] [per_page] [--access_token] [token]
```

参数说明：

- check\_items: 指定检测项，以逗号分隔，为空表示所有
- host\_list: 为指定主机列表，以逗号分隔，为空表示所有
- start: 指定异常检测的起始时间，格式类似 20200101-11:11:11，不指定时默认以截止时间往前推 1 小时
- end: 指定异常检测的截止时间，不指定时默认为当前时间
- sort: 对异常检测结果进行排序，目前可选 start 或 end 或 check\_item，为空表示不排序
- direction: 异常结果排序可选升序(asc)，降序(desc)，默认:asc
- page: 当前的页码
- per\_page: 每页的数量，最大为 50
- access\_token: 访问令牌

3. 举例，获取主机 host1 上 check\_item1 在 2021-9-8 11:24:10~2021-9-9 09:00:00 的异常检测结果，仅获取第 1 页，每页显示 30 个，按 check\_item 升序排序，：

```
adoctor check --check_items check_item1 --host_list host1 --start 20210908-11:24:10 --end 20210909-09:00:00 --sort check_item --direction asc --page 1 --per_page 30 --access_token "1111"
```

## 10 日志查看与转储

1. adoctor-check-scheduler 日志
  - 路径：/var/log/aops/uwsgi/check\_scheduler.log
  - 功能：主要记录代码内部运行的日志，方便定位问题
  - 权限：路径权限 755，日志权限 644，普通用户可以查看
  - debug 日志开关：/etc/aops/system.ini 中 log\_level 配置项
2. adoctor-check-executor 日志
  - 路径：/var/log/aops/aops.log
  - 功能：主要记录代码内部运行的日志，方便定位问题
  - 权限：路径权限 755，日志权限 600，普通用户可以查看
  - debug 日志开关：/etc/aops/system.ini 中 log\_level 配置项